ADLC

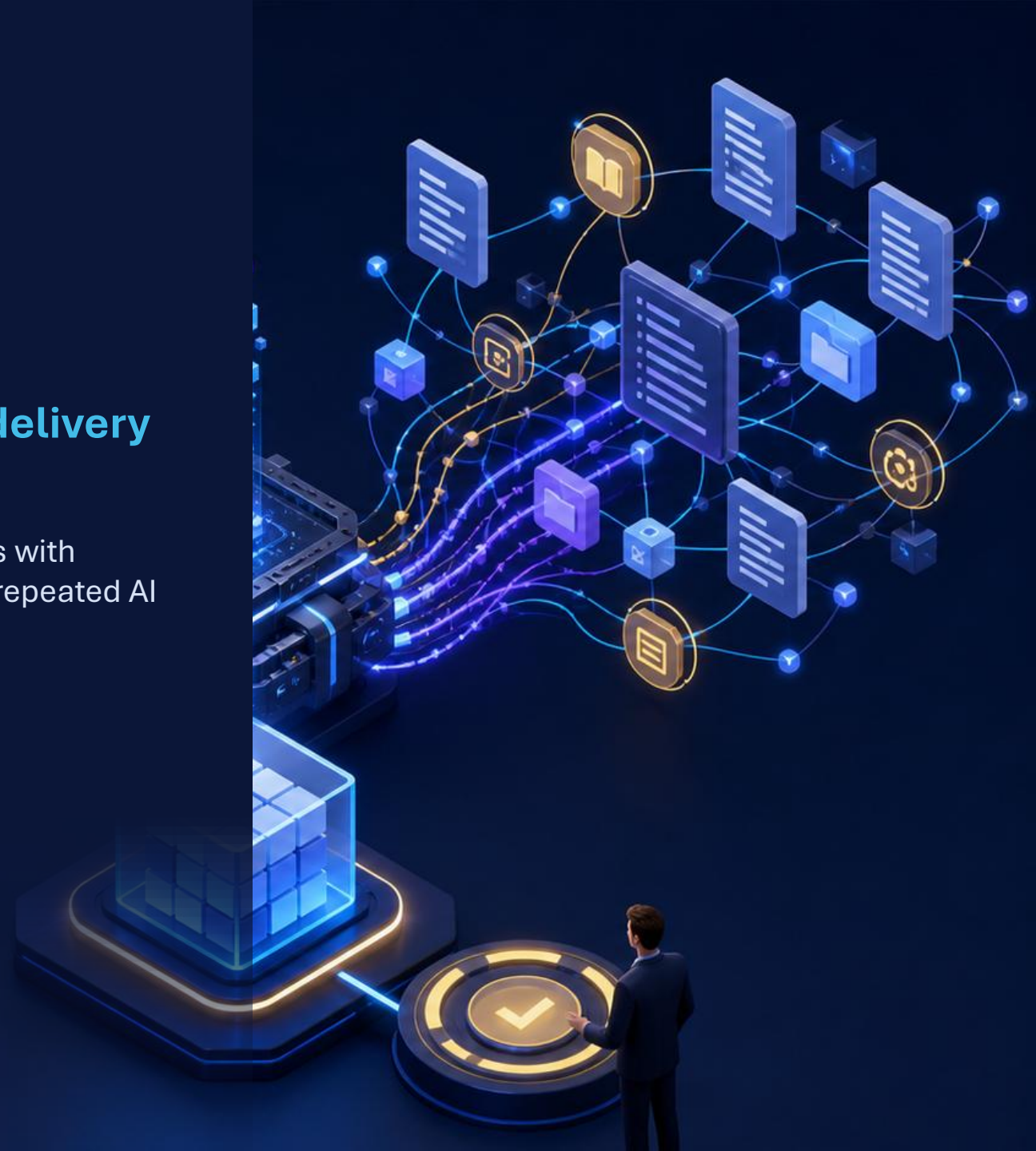
PROJECT PROPOSAL

ADLC Studio

Pattern-first, human-governed application delivery

A four-module platform that combines reusable code templates with project-aware retrieval to generate only what is new—reducing repeated AI work, cost and delivery variation.

DOCUMENT | DESIGN | DEVELOP | DEPLOY



Created by NIKHIL SHINDE

Leadership review • July 2026

WHY NOW

The Compounding Delivery Tax of Enterprise LLMs

Generative AI accelerates isolated tasks, but uncontrolled usage creates a new class of cost, latency and governance problems.

The Reality of Unfiltered AI Integration

Applying massive Large Language Models to every stage of software development introduces a severe and compounding **delivery tax**. Organizations pay premium computational costs for LLMs to continuously rethink and regenerate solutions for already solved, deterministic problems.
This unfiltered usage drives up token consumption, leading to unsustainable cloud costs, increased system latency, and a dangerous vendor dependency that locks lifecycles into proprietary models instead of leveraging reusable intelligence.

The core question

How do we preserve AI speed without paying an AI tax on every step?

Repeated reasoning

The model re-discovers project structure, naming, standards and conventions on every request.

Token-heavy workflows

Full codebase context is repeatedly sent even when a deterministic rule or cached pattern is enough.

Inconsistent outputs

Different prompts and model responses create drift across documents, code, tests and diagrams.

Human review burden

Teams spend time validating generated artifacts without an unbroken requirement-to-release chain.

CORE INNOVATION

Remove repeated AI work—not human judgment

ADLC uses the least expensive, most predictable intelligence that can safely complete each task.

Automate the repeatable

- Project fingerprinting and file classification
- Naming, folder, template and policy enforcement
- Trace links and impact propagation
- Cached plans, prompts and accepted patterns
- Deterministic validation and release checks

PATTERN-FIRST

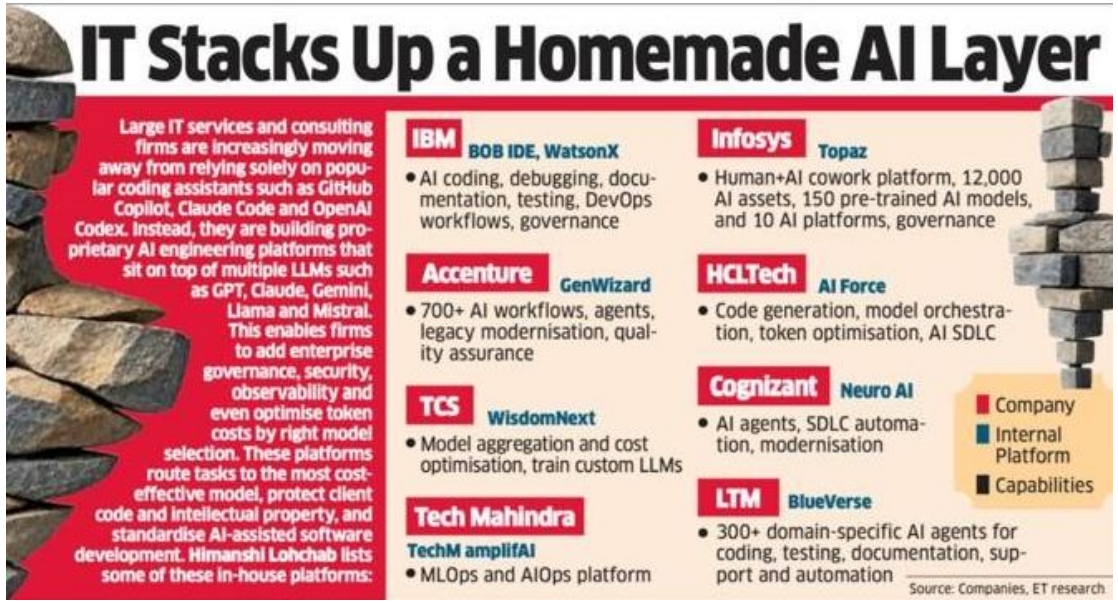
Keep people in command

1. Approve requirements and architecture baselines
2. Review diffs before files are applied
3. Resolve ambiguous or high-risk changes
4. Sign off tests, controls and deployment
5. Override policies with an auditable reason

MARKET CONTEXT

Enterprise AI platforms are becoming the new delivery layer

Large IT services firms are building proprietary AI stacks around coding, testing, governance and model orchestration.



Observed platform landscape

TCS
WisdomNext

Infosys
Topaz

HCLTech
AI Force

Accenture
GenWizard

LTM
BlueVerse

Tech Mahindra
amplifAI

ADLC combines reusable code templates with project-aware RAG to generate only what is new—reducing model cost while maintaining human control and end-to-end lifecycle traceability.

ADLC whitespace

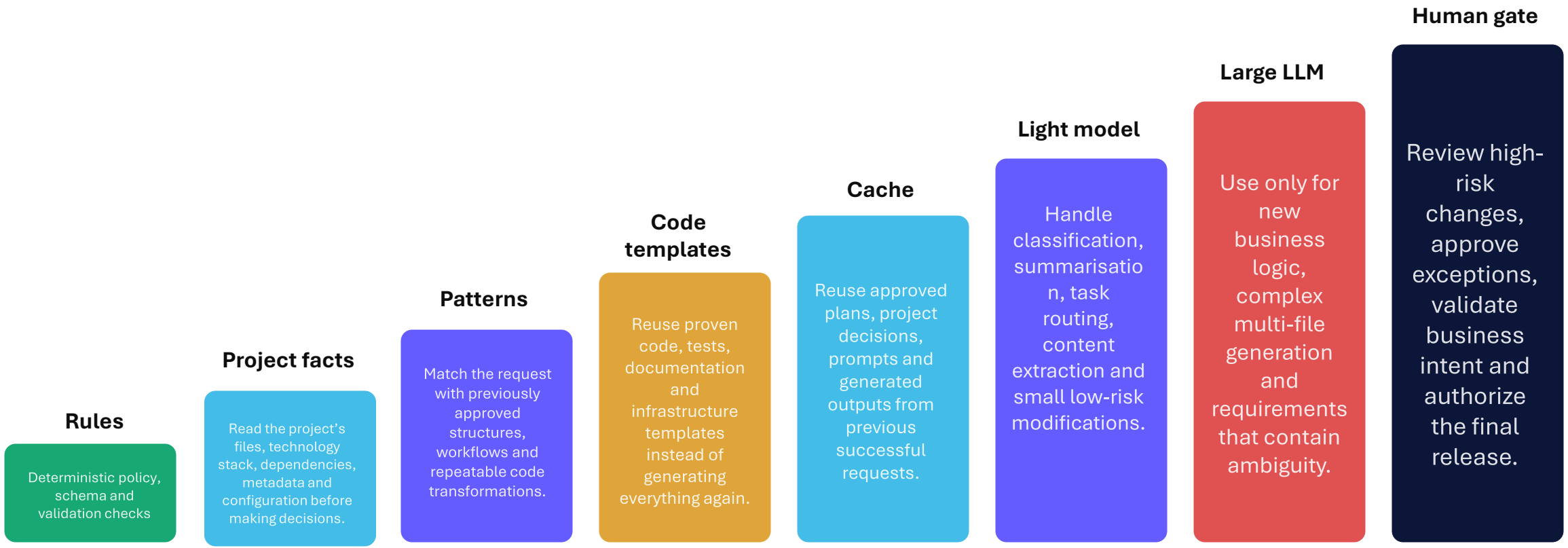
Pattern reuse + token governance + lifecycle traceability in one builder workspace.

Platform names are drawn from the user-provided market reference and shown only as competitive context.

DESIGN PRINCIPLE

The Least-Intelligence-First ladder

Every request climbs only as high as needed—reducing latency, cost and output variability.



LOWER COST • LOWER LATENCY • HIGHER PREDICTABILITY

ESCALATE ONLY WHEN VALUE JUSTIFIES IT

PLATFORM SCOPE

Four modules, one shared project intelligence layer

Each module is useful independently; together they create a continuously traceable delivery system.

01

ADLC Documentation

Requirements, BRD, ADRs, API guides, runbooks

02

ADLC Designer

Architecture, data, sequence and deployment diagrams

03

ADLC Development

Project plans, code generation, refactor and tests

04

ADLC Deployment

IaC, CI/CD, manifests, release and rollback plans

SHARED PROJECT INTELLIGENCE

Project fingerprint | Pattern library | Knowledge index + RAG | Traceability graph | Policy + cost router

MODULE 1

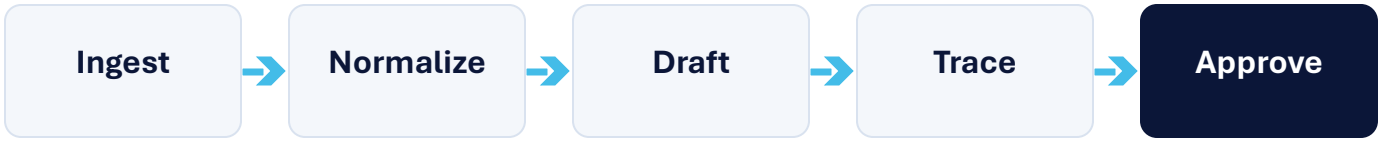
ADLC Documentation

Generate and maintain project documents from the same facts and trace links used by design and code.

DOC

From scattered inputs to a governed baseline

Ingest prompts, meeting notes, schemas, existing documents and source code. Normalize them into structured, versioned deliverables with review gates.



Differentiator: update impacted sections only—do not regenerate the entire document.

MVP deliverables

- Business / product requirement document
- Architecture decision records
- API and data dictionary
- Test and acceptance plan
- Operations runbook + release notes

MODULE 2

ADLC Designer

Create diagrams as editable architecture models—not static AI images disconnected from the project.

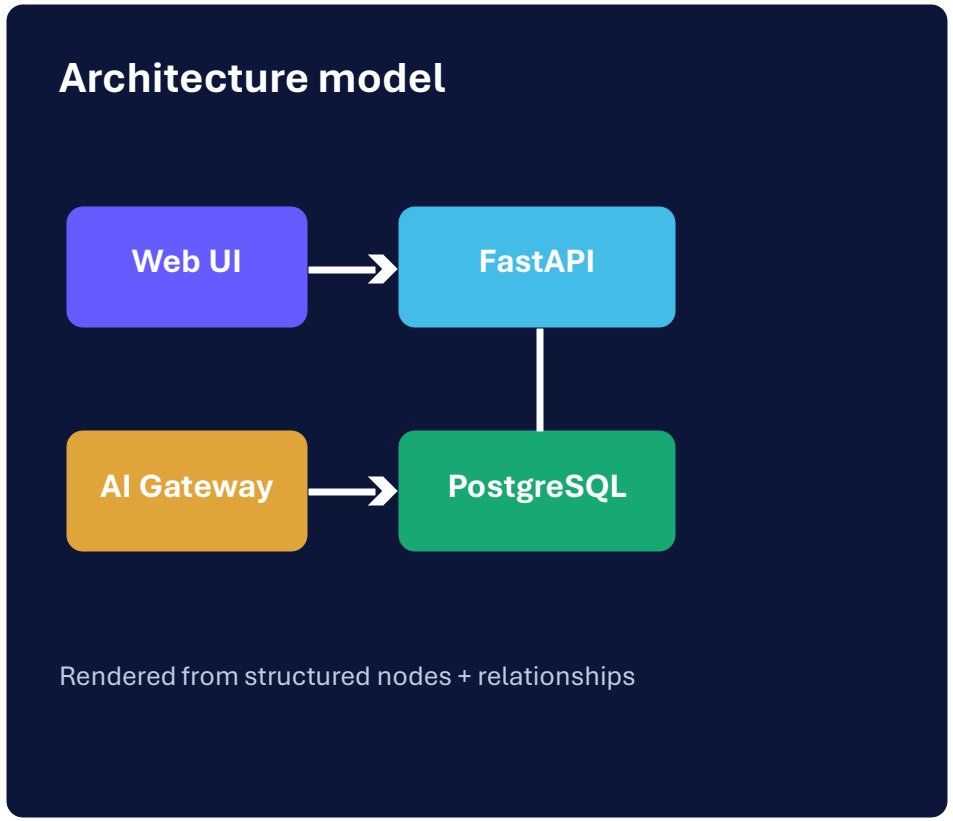


Diagram families

System context
People, systems and trust boundaries

Container / component
Services, modules and ownership

Data model
Entities, keys and relationships

Sequence
Runtime flow and error paths

Deployment
Environments, infra and network

MODULE 3

ADLC Development

Extend the current workspace into a governed code-generation and modification system.

1

Understand

Fingerprint stack, files and patterns

2

Plan

Propose files, dependencies and acceptance checks

3

Generate

Reuse templates; call LLM only for novel sections

4

Preview

Show diff, rationale, cost route and impacted traces

5

Apply

Persist atomically, index and record history

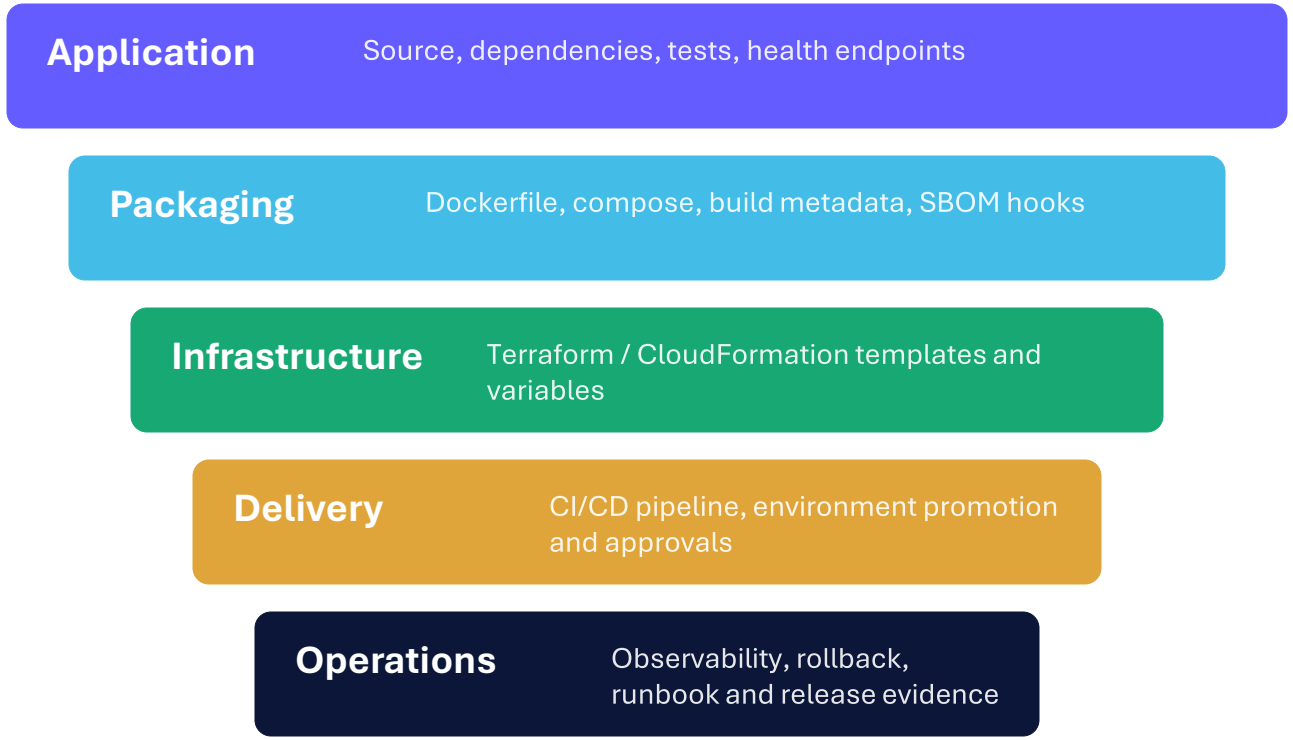
Current prototype foundation

- **Project wizard**
- **Physical workspace**
- **Editor + autosave**
- **Preview / apply / reject**
- **Generation history**
- **RAG index + chat**

MODULE 4

ADLC Deployment

Turn an approved project baseline into environment-ready delivery assets with explicit release gates.



Guarded release flow

1. Validate configuration
2. Build + test
H. **Human approval**
4. Deploy
5. Verify + record

No autonomous production deployment in the MVP.

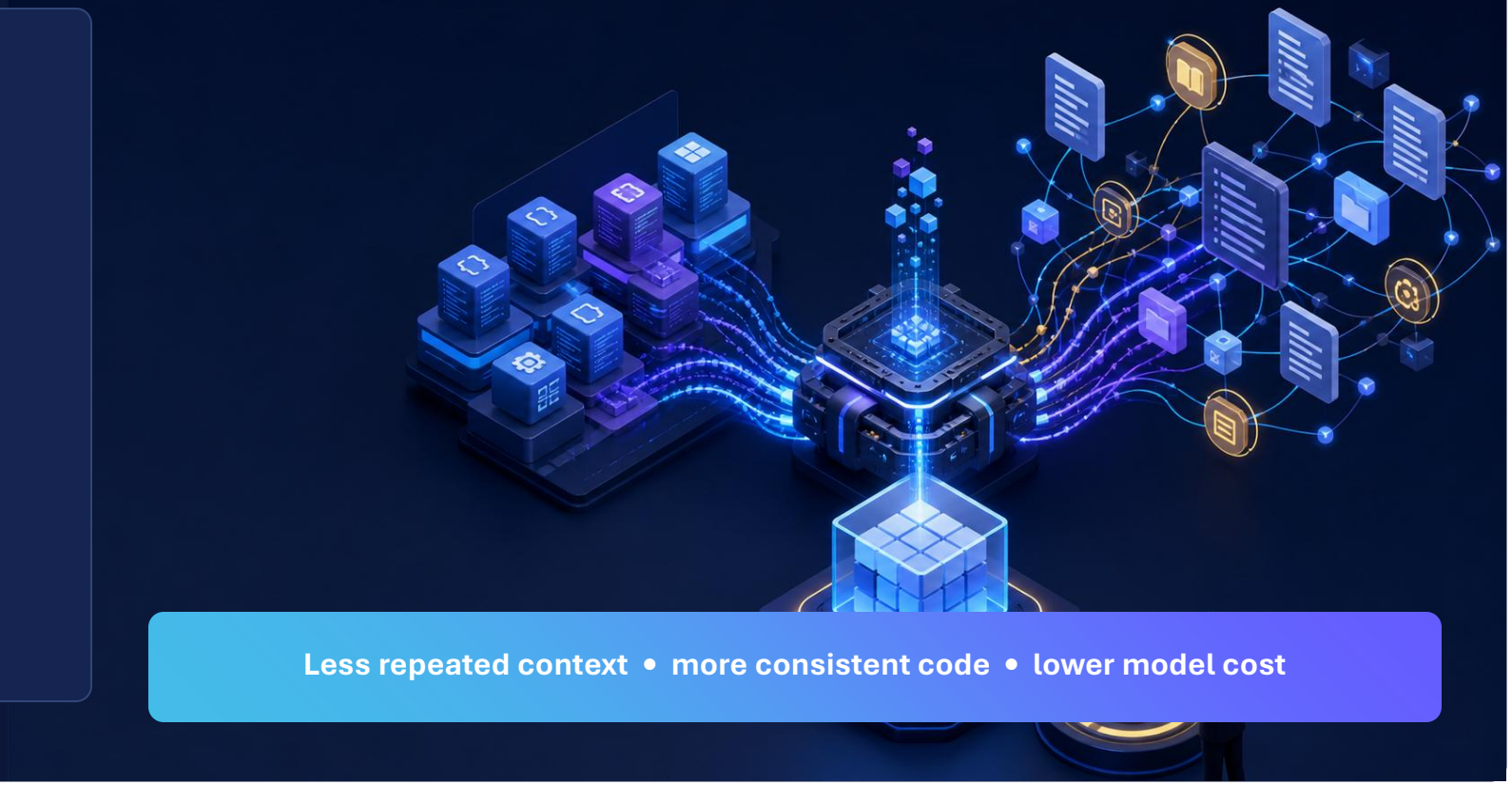
CORE INNOVATION

Code Templates + RAG generate only what is new

Reusable structure supplies the foundation; project-aware retrieval supplies the facts; focused synthesis fills only the remaining gaps.

Reuse-first generation

1. **Code Templates**
   Create proven project structure, routes, models, tests and configuration.
2. **RAG Retrieval**
   Fetch only the relevant project files, documents and accepted examples.
3. **Focused Synthesis**
   Use the AI Model Gateway only for genuinely new logic or ambiguity.
4. **Validate + Approve**
   Run checks, show the diff and require human approval before apply.

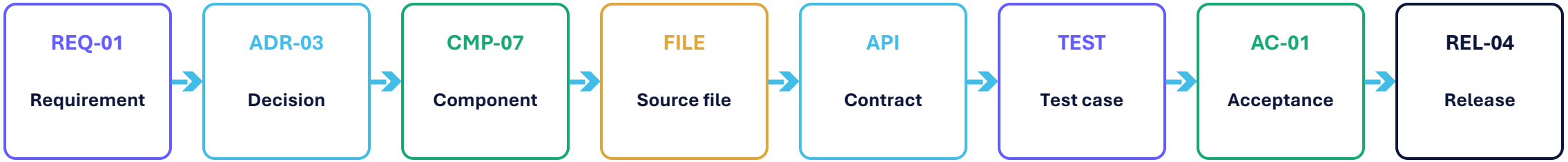


Less repeated context • more consistent code • lower model cost

GOVERNANCE BY DESIGN

An unbroken requirement-to-release traceability chain

Every generated artifact carries forward the reason it exists and the evidence that it is acceptable.



Impact analysis becomes a query—not a meeting

Example: changing REQ-01 identifies the affected design decision, component, files, API contract, tests, acceptance criteria and release notes before any code is applied.

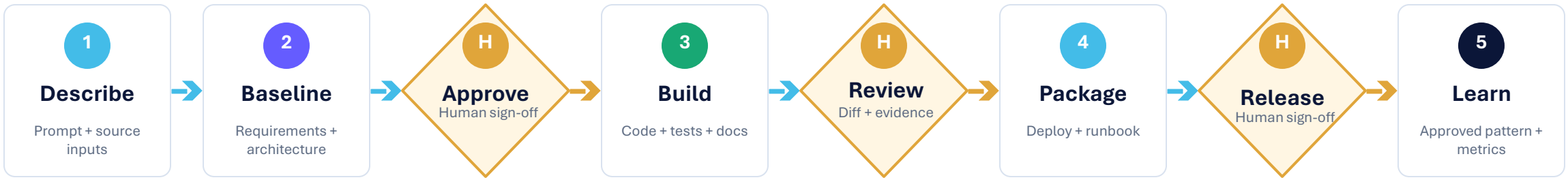
Measured
Trace coverage

Flagged
Orphan artifacts

OPERATING MODEL

Human-governed flow from intent to release

ADLC automates construction between explicit decision gates, with a durable audit record at every transition.



AUDIT RECORD

- Who requested
- What changed
- Why routed
- Model / pattern used
- Cost + latency
- Who approved
- Evidence produced

COST AND PERFORMANCE

Make intelligence routing measurable

The MVP should expose why each task used rules, patterns, a light model or a large LLM.

Per-run decision card

Task	Modify product search filter
Context	4 files • 2 trace nodes
Route	Pattern → light model validation
Reason	Accepted CRUD/filter pattern matched
Fallback	Large LLM only if validation fails

Leadership metrics

Pattern reuse rate

How often approved patterns avoid new generation

LLM escalation rate

Share of tasks that require a large model

Cost per accepted change

Spend normalized by applied, approved output

Time to approved change

Request-to-apply duration including review

Rework / rejection rate

Previewed changes rejected or regenerated

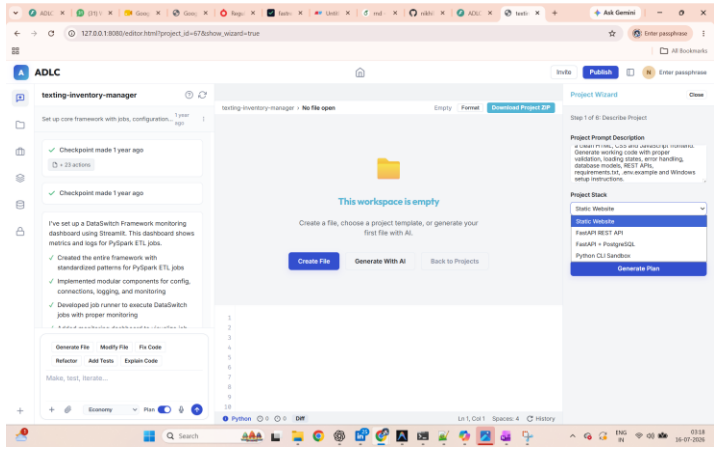
Trace coverage

Released artifacts connected to requirement and test

PROTOTYPE EVIDENCE

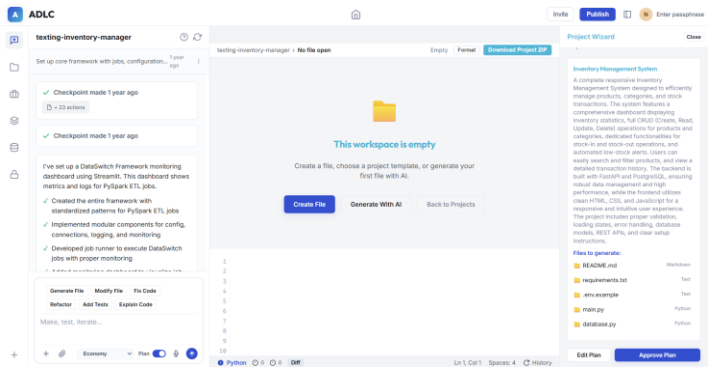
A working foundation already exists

The current ADLC prototype demonstrates the workspace, planning and code-change interaction model needed for the Development module.



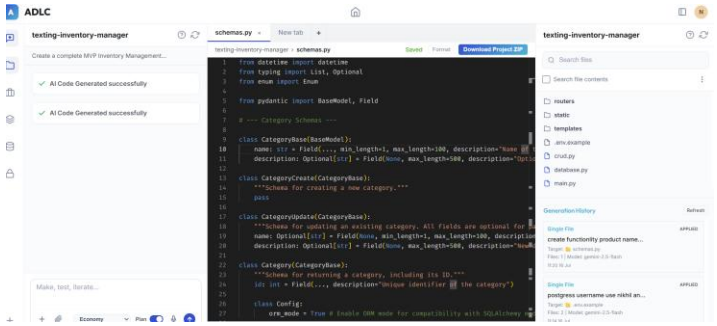
1 • Describe + select stack

Project wizard captures intent and recommends a stack.



2 • Review generated plan

Plan preview lists files before generation begins.



3 • Edit + review history

Persistent files, syntax-aware editor and generation audit trail.

Existing capabilities: FastAPI • PostgreSQL + pgvector • AI Model Gateway • session auth • physical workspaces • preview/apply/reject • RAG

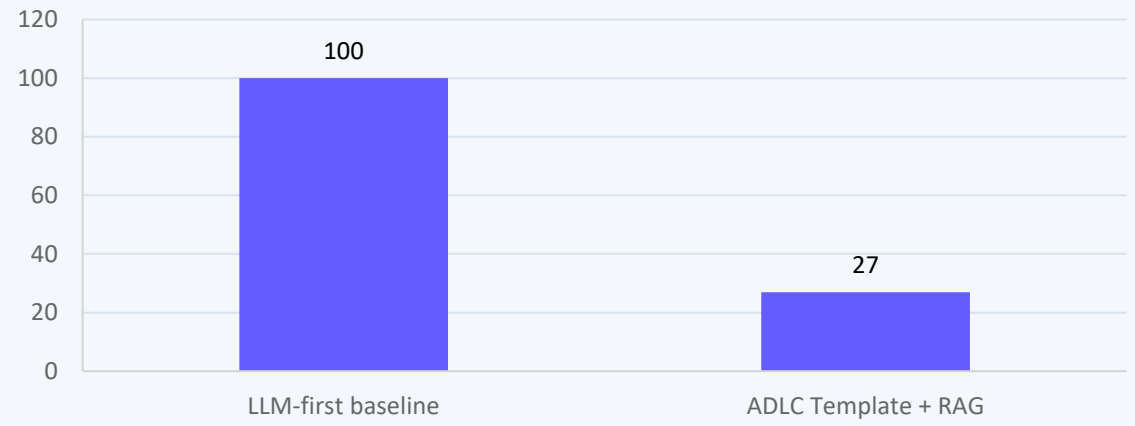
Source: current ADLC prototype screenshots supplied by the project owner.

ILLUSTRATIVE COST MODEL

Template + RAG routing can reduce model cost by about 73%

The comparison uses a normalized cost index—not provider pricing—and must be replaced with pilot telemetry before any external claim.

Normalized monthly model-cost index



Baseline = 100 cost units

73% illustrative saving

If current model spend is

₹1,00,000 / month

ADLC illustrative spend

₹27,000 / month

SAVE ₹73,000 / MONTH • ₹8.76 LAKH / YEAR

Scenario assumptions

60% template execution • 25% RAG-grounded focused generation using 25% of baseline context • 15% full synthesis • retrieval/indexing overhead included

ROADMAP

Build value in four evidence-driven increments

Each phase delivers usable capability and creates the data needed to justify the next investment.

Stabilize Development MVP

- Latency + failure instrumentation
- Project isolation regression suite
- Generation route metadata

PHASE 1

Pattern + trace foundation

- Project fingerprint
- Pattern registry + cache
- Requirement-to-file/test links

PHASE 2

Documentation + Designer

- Incremental document updates
- Structured diagram model
- Cross-module impact analysis

PHASE 3

Deployment pilot

- Docker/IaC/pipeline generation
- Release evidence + runbook
- Human-gated non-prod deployment

PHASE 4

Gate to proceed: measured reuse, trace coverage, accepted-change time and model escalation—not slideware completion.

DECISION REQUIRED

Approve a focused ADLC Studio pilot

Use one representative internal application to validate the pattern-first operating model end to end.

Pilot scope

1	**1 application**	Existing codebase with real change requests
2	**2–3 repeatable patterns**	For example CRUD, API contract and deployment baseline
3	**3 human gates**	Baseline, change approval and non-prod release
4	**4 modules demonstrated**	Documentation, design, development and deployment

Leadership ask

- ✓ Name an executive sponsor
- ✓ Provide one pilot product team
- ✓ Approve architecture + delivery SMEs
- ✓ Permit instrumented model usage
- ✓ Review pilot evidence at phase gates

Decision: fund the pilot, establish the baseline, and let measured evidence determine scale.

APPENDIX

Assumptions, success measures and open decisions

A transparent boundary between what exists today, what is proposed, and what the pilot must prove.

Foundation Available Today

- FastAPI backend and API services
- PostgreSQL with pgvector knowledge store
- Vendor-neutral AI Model Gateway
- Session-based authentication
- Project-isolated physical workspaces
- Editor, autosave and file management
- Plan, preview, apply, reject and history
- Project indexing and RAG assistance

Capabilities to Complete

- Documentation module
- Structured architecture designer
- Deployment asset generator
- Project fingerprint
- Qualified pattern registry
- Least-intelligence router
- Traceability graph
- Cost and performance telemetry

Pilot Decisions Required

- Pilot application and owner
- Approved model portfolio
- Data and code handling boundary
- Required human gates
- Pattern qualification criteria
- Non-production target environment
- Success thresholds
- Go / no-go review cadence

No savings, productivity or autonomy claims should be externalized until the pilot establishes measured evidence.

Conceptual inputs include the user-provided management reference slides and current ADLC prototype evidence.